

Assignment 2

Youssef Hossam Eldeen Mohamed Elzainy Mostafa - 212600

Part one

Task one

Task 1: Identifier and Case Sensitivity Analysis

Determine whether the following identifiers are Valid or Invalid. If invalid, briefly explain why.

1. data_bus_32
2. \$display_enable
3. _cdc_sync_01
4. \cpu_reset_n (Note the trailing space)
5. SoC-AXI-Interface
6. wire_internal-sig
7. module
8. Clock_50MHz
9. \display_val
10. 1st_data_line
11. \reset#_n
12. \$display_val
13. DATA and data (Are these considered the same variable?)

1. Valid
2. Invalid - started the identifier with a dollar sign (\$)
3. Valid
4. Valid
5. Invalid - used the dash sign (-) in between the words
6. Invalid - used the dash sign (-) in between the words
7. invalid - module is a keyword and can't be used for identifiers
8. Valid
9. Valid
10. Invalid - started the identifier with a number
11. Valid
12. Invalid - started the identifier with a dollar sign (\$)

13. Not the same variable because Verilog is a case sensitive HDL

Task two

Task 2: Number Formats and Logic Values

Translate the following descriptions into standard Verilog literal integers or real numbers.

1. A **16-bit hexadecimal** number with the value **FF0A**.
2. An **8-bit binary** number where the **upper 4 bits** are **unknown (x)** and the **lower 4 bits** are **high-impedance (z)**.
3. An **unsized decimal** number representing **4096**. (two ways)
4. A **4-bit binary** number representing **-5**.
5. An **unsized decimal** number representing **-4096**. (two ways)
6. A **real number** representing **1.25×10^{-4}** .
7. An **unsized decimal** number representing **the first 15 digits of pi (π)** after the decimal point.
8. A **4-bit octal** number representing **10 in decimal**

1. `16'hFF0A`
 2. `8'bxxxx_zzzz`
 3. `'d4096`
 4. `-4'sb0101`
 5. `- 'd4096`
 6. `1.25e-4`
 7. `'d141592653589793`
 8. `4'o12`
-

Task three

Verilog Literal	Bit Width	Radix / Base	Binary Equivalent	Decimal Value
8'hA3				
6'h2x				
4'b101z				
12'o47				
8'd255				
'h1F				
-15				
325				
5'sd10				
3'sd12				
-6'd7				
10'bx0_101z				
5'shC				

Verilog Literal	Bit Width	Radix / Base	Binary Equivalent	Decimal Value
8'hA3	8	hexadecimal	1010 0011	163
6'h2x	6	hexadecimal	10 xxxx	unknown value
4'b101z	4	binary	101z	high impedance
12'o47	12	octal	000 000 100 111	39
8'd255	8	decimal	1111 1111	255
'h1F	32	hexadecimal	0001 1111 (lower 8 bits) the rest are all zeros	31
-15	32	decimal	10001 lower 5 bits the rest are all ones	-15
325	32	decimal	101000101 lower 9 bits the rest are all zeros	325
5'sd10	5	signed decimal	01010	10
3'sd12	3	signed decimal	1 100 → the MSB is truncated	-4
-6'd7	6	unsigned decimal	111001	57
10'bx0_101z	10	binary	xxxxx0101x	unknown
5'shC	5	signed hexadecimal	01100	12

Part two

Part 2: Lab Activity (Hands-On)

- **Objective:** Identify and **resolve intentional lexical errors** within a provided **Verilog module**.
- **Scenario:** You have received an RTL file for a simple reset synchronizer—a critical component used in Clock Domain Crossing (CDC) to safely bring an asynchronous reset into a local clock domain. However, the engineer who wrote the code made several critical lexical mistakes that will prevent compilation.
- **Task:**
 1. Review the **.v** code below.
 2. Identify **all lexical errors**.
 3. **Rewrite the module to compile successfully.**

```
Module 1st_reset_sync (
    input wire clk,
    input wire async_rst_n,
    output reg sync_rst_n
);
    /* This block comment explains the module
    /* The clock is active high */
    and reset is active low
    */
    // Internal registers for the 2-stage synchronizer
    reg meta_reg;
    (* preserve reg sync_reg;

    // Define a delay parameter using an invalid base format
    parameter DELAY = 8'd 15;
    parameter MAX_VAL = 4'b1021;

    always @(posedge clk or negedge async_rst_n) begin
        if (!async_rst_n) begin
            Meta_reg <= 1'b0;
            Sync_reg <= 1'b0;
        end else begin
            Meta_reg <= 1'B1;
            Sync_reg <= Meta_reg;
        end
    end

    /* Drive the output
    assign sync_rst_n = sync reg;
end module
```

XX

Module 1st_reset_sync (

// keyword is module with now upercases

// module name is invalid - can't start with a number

XX

```
    input wire clk,
    input wire async_rst_n,
    output reg sync_rst_n
```

);

XX

/* This block comment explains the module

ection

```
/* The clock is active high */
and reset is active low
*/
//-----
// the first error is that he tried to do a nested comment s

//-----
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

// Internal registers for the 2-stage synchronizer
reg meta_reg;

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
(* preserve reg sync_reg;
// unclosed attribute
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

// Define a delay parameter using an invalid base format

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
parameter DELAY = 8'd_15;
// decimal started with an underscore which is invalid
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
parameter MAX_VAL = 4'b1021;
// wrote the "2" digit in binary which is invalid
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

always @(posedge clk or negedge async_rst_n) begin
    if (!async_rst_n) begin

        Meta_reg <= 1'b0;
        Sync_reg <= 1'b0;

    end else begin

        Meta_reg <= 1'B1;
```

```

        Sync_reg <= Meta_reg;

    end

end
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
/* Drive the output
unclosed comment block */
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
assign sync_rst_n = sync_reg;
// can't use continous assignment with a reg
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

end module

```

correct version

```

module reset_sync (
    input wire clk,
    input wire async_rst_n,
    output reg sync_rst_n
);

/* This block comment explains the module
The clock is active high
and reset is active low
*/
// Internal registers for the 2-stage synchronizer

reg meta_reg;
(* preserve *) reg sync_reg;

parameter DELAY = 8'd15;
parameter MAX_VAL = 4'b1010;

always @(posedge clk or negedge async_rst_n) begin
    if (!async_rst_n) begin
        Meta_reg <= 1'b0;
        sync_reg <= 1'b0;
    end
end

```

```

    end else begin
        Meta_reg <= 1'b1;
        Sync_reg <= meta_reg;
    end
end

/* Drive the output */
always @(*) begin
    sync_rst_n = sync_reg;
end

endmodule

```

Part three

```

/*
Lab: Verilog Lexical Conventions
Description: Demonstrates identifiers, numbers, strings, and attributes.
*/
// TODO: Add a module-level attribute here (e.g., (* optimize = "off"
*))
module lexical_demo;
// TODO:
// 1. Strings
// A string requires 8 bits per character. "Hello World" is 11 characters.
// Declare a register named 'my_string'
reg [87:0] my_string;
// 2. Case Sensitivity & Identifiers
// Declare two 8-bit registers named 'Data_Val' and 'data_val'
reg [7:0] Data_Val, data_val;
// 3. Logic Values & Literal Integers
// Declare a 12-bit register named 'mixed_logic'
reg [11:0] mixed_logic;
// 4. Literal Real Numbers
// Declare a 'real' variable named 'analog_voltage'
real analog_voltage;
// 5. Attributes on variables
// use (* keep = "true" *) to a net called 'protected_wire'

```

```

(* keep = "true" *)
wire protected_wire;

initial begin
    // --- Assignments ---
// TODO:
    // String assignment

    // TODO: assign "Hello World" to my_string
    my_string = "Hello World";

    //-----
--

    // TODO: Assign 8'hAA to 'Data_Val' and 8'h55 to 'data_val'

    Data_Val = 8'hAA;
    data_val = 8'h55;

    // TODO: Assign a 12-bit binary number to 'mixed_logic' (1, 0,
x, z)
    mixed_logic = 12'b111_000_xxx_zzz;
    // Example: 12'b1010_xxxx_zzzz;

    //-----
--

    // TODO: Assign a real number to 'analog_voltage' (e.g., 3.3e-3)

    analog_voltage = 3.3e-3;

    // --- Display Results ---
    // $display statements automatically print to the console.
    $display("--- Lexical Conventions Output ---");
    $display("String Value: %s", my_string);
    // TODO: Add $display statements to print variables.

    $display("Data_Val: %h", Data_Val);
    $display("data_val: %h", data_val);

```

```

$display("Mixed Logic (binary): %b", mixed_logic);
$display("Mixed Logic (hex): %h", mixed_logic);

$display("Analog Voltage: %f", analog_voltage);
// Hint: Use %h for hex, %b for binary, and %f for real numbers.
$finish;

end
endmodule

```

	Msgs
+ my_string	88'aHello World
+ Data_Val	8'haa
+ data_val	8'h55
+ mixed_logic	12'b111000xxxzzz
analog_voltage	0.0033
protected_wire	1'hz

```

V$IM5> run -all
# --- Lexical Conventions Output ---
# String Value: Hello World
# Data_Val: aa
# data_val: 55
# Mixed Logic (binary): 111000xxxzzz
# Mixed Logic (hex): eXX
# Analog Voltage: 0.003300
# ** Note: $finish      : D:/NTI - Digital IC Design/Session 3/Assignment/Lexical demo.v(67)
#   Time: 0 ns  Iteration: 0  Instance: /lexical_demo
# 1
# Break in Module lexical_demo at D:/NTI - Digital IC Design/Session 3/Assignment/Lexical demo.v line 67

```